SPARXSTAR

SOVEREIGN KNOWLEDGE FRAMEWORK

SPARXSTAR

Sovereign Knowledge Framework

Platform Overview & Repository Map

Starisian Technologies · Version 1.0

This document is the entry point for all developers working on this platform.

Read this before any specification document. Read the Platform Integrity Map before any repository.

CONFIDENTIAL · PATENT PENDING

What This Platform Is

There is a woman sitting in a room. She is old. She knows things that no living person knows — words, stories, the names of plants that heal, the reason a certain bird appears before rain, the riddle her grandmother told that contains, encoded in its answer, a number that describes the movement of stars.

She will die. When she dies, what she knows dies with her.

SPARXSTAR is the system built to receive what she gives, hold it exactly as she gave it, on terms she and her community control, and make it useful for her people — without ever allowing it to be taken, used, or ingested by anyone without explicit sovereign release.

That is the why. Every technical decision in every repository flows from it.

The platform is also a commercial content engine. The same architecture that protects an elder's oral history protects an artist's unreleased recording, a tribe's governance records, and a musician's campaign brief. The Digital Village Elder and the Press Room are the same engine aimed at different purposes. Build the commercial product. The Elder is already inside it.

The Platform Stack — A Hero Journey in Architecture

Data does not simply move from input to database. It ascends through a sequence of thresholds. Each layer is a threshold. Each threshold is earned. No data reaches the vault without passing every layer in order.

Layer	Name	Mythological Role	Technical Role	Schema Loaded
∞	Ouroboros Integrity	The Cosmic Serpent — devours its own tail. Loads first, enforces last, present at every boundary always.	Execution substrate. Shared exceptions, contracts, DTOs, loader, platform constants. Every repo depends on this. Nothing else loads without it.	None — enforces contracts
☀️	Helios Trust	The Sun — trusted, constant, always rising. The unchanging foundation of trust.	Edge Agreement Engine. Stateless fail-closed evaluator. Decides whether a specific request may proceed.	None — evaluates trust
✦	Sirus Context	The North Star — fixed in the sky, the navigator's anchor. Guides all motion beneath it.	Sovereign context engine. Establishes who is present, on what device, under what authority, before anything else executes.	None — produces context
☁️	Sky DVE Core	The Air — where prayers are sent, where secrets are whispered. Sky hears our words.	AI intake layer. Converts conversation to structured draft. Extracts meaning without inventing it. Waits for human commit.	SCF schema + Block schema (Abilities)
🌙	Mḗh₁n̥s DVE Core	The Moon — the measurer of time, the secret-keeper. Aligns what the air carries against law and protocol.	The Epistemic Sieve. Governance enforcement, policy evaluation, cultural law. Aligns the payload before it returns to earth.	Policy schema (JSON governance grammar)
🌍	Dheghom DVE Core	The Earth — where all things return. The Digital Village Elder who holds the memory of the people.	The schema-driven vault. Modular DAL. Persistent sovereign storage. The final resting place of the aligned, governed truth.	SCF schema only

The execution order is always: Ouroboros → Helios → Sirus → Sky → Mḗh₁n̥s → Dheghom. This order is enforced by mu-plugins/00-sparxstar-loader.php. Nothing loads outside this sequence. If the loader fails, the system does not exist.

Why Ouroboros

The Cosmic Ouroboros is a theoretical concept in cosmology describing the connection between the smallest scales of the universe — Planck length, the quantum world — and the largest scales — the entire universe. It is depicted as a serpent devouring its own tail: the extremes meet, the beginning and the end are the same point, the cycle is complete and unbreakable.

Ouroboros Integrity loads at the very beginning of every request. It enforces its contracts at every boundary throughout execution. It is still present and enforcing at the very end. Every other component — Helios, Sirus, Sky, Méh₁ṇs, Dheghom — depends on it and cannot exist without it. It is the substrate on which everything else runs.

The name is exact. It is not decorative.

The repository is named sparxstar-ouroboros-integrity rather than sparxstar-ouroboros because "integrity" is the technical meaning of what it enforces: the Platform Integrity Map made physical. The name distinguishes the running code from the document that defines its rules.

Repository Map

All repositories are under the Starisian Technologies GitHub organization. They are currently private. Each repository has its own complete specification document. The Platform Integrity Map governs all of them.

Repository	URL	Load Order
sparxstar-ouroboros-integrity	sparxstar-ouroboros-integrity	0 — loads first, always
sparxstar-helios-trust	sparxstar-helios-trust	1 — Edge + Origin gate
sparxstar-sirus-context	sparxstar-sirus-context	2 — Context kernel
sparxstar-sky-dve-core	sparxstar-sky-dve-core	3 — Intake layer
sparxstar-mehns-dve-core	sparxstar-mehns-dve-core	4 — Governance sieve
sparxstar-dheghom-dve-core	sparxstar-dheghom-dve-core	5 — The vault
sparxstar-digital-village-elder-schema	sparxstar-digital-village-elder-schema	Schema reference (not a runtime component)

Schema Repository

sparxstar-digital-village-elder-schema is not a runtime component. It is the canonical reference for what a conforming SCF schema looks like. The temporal-cortex branch contains a working example of the SCF schema shape that Dheghom and Sky load.

This repository defines the shape, not the limit. Dheghom loads any schema conforming to the SCF pattern. Sky Abilities load any schema conforming to the SCF or Block pattern. Mḗh₁n̥s loads any schema conforming to the SPARXSTAR Policy grammar. The DVE schema repo is the reference example — not the only schema the platform will ever serve.

Schema Loading Model

The platform supports three schema types. Each layer loads the schema types appropriate to its function. Schema loading is not interchangeable — Dheghom does not load Policy schemas, Mḗh₁n̥s does not load Block schemas, and neither loads the other's schema type.

SCF Schema (Secure Custom Fields)

Loaded by: Dheghom DVE Core and Sky DVE Core (Abilities).

Purpose: defines the field structure of governed content records — what fields exist, their types, which are required for release, which are asset fields, and how they map to the Dheghom module registry.

Shape: the canonical example is in sparxstar-digital-village-elder-schema, branch max/schemas/canonical/temporal-cortex. Any schema following this shape is valid. The platform loads all conforming SCF schemas — not only the DVE schema.

- Dheghom uses SCF schemas to define its 57 module field groups and their validation rules
- Sky uses SCF schemas to define Abilities — the structured extraction targets the AI maps conversation to

Block Schema

Loaded by: Sky DVE Core (Abilities only).

Purpose: defines WordPress block-based content structures that Sky can extract into. Allows Sky to produce content shaped for the WordPress block editor as a direct output target, in addition to SCF-defined structured records.

- Block schemas are loaded alongside SCF schemas in Sky Abilities
- Block schemas are not loaded by Dheghom — Dheghom stores governed structured records, not block editor output

Policy Schema (SPARXSTAR JSON Policy Grammar)

Loaded by: Mḗh₁n̥s DVE Core only.

Purpose: defines governance rules in the SPARXSTAR JSON policy language — the IF/THEN rule grammar that the Epistemic Sieve evaluates at release time. Policy Packs are collections of policy schema files.

The Policy schema is a proprietary grammar defined in the Mḗh₁n̥s specification. It is not an SCF schema and not a Block schema. It is the constitutional law layer of the platform.

- Mḗh₁n̥s loads Policy schemas from /policies/global/, /policies/authority/, /policies/ability/, and /policies/user/
- Policy schemas are never loaded by Dheghom or Sky

Schema Loading Summary

Component	SCF Schema	Block Schema	Policy Schema
Ouroboros Integrity	No	No	No
Helios Trust	No	No	No
Sirus Context	No	No	No
Sky DVE Core — Abilities	Yes	Yes	No
Méh₁n̥s DVE Core	No	No	Yes
Dheghom DVE Core	Yes	No	No

The DistroKid Mental Model

For non-technical stakeholders and new developers, the platform works exactly like a music distribution service. This analogy is precise enough to use in conversation.

Step	Music Analogy	SPARXSTAR Equivalent
1	Set up Artist profile	Helios Trust + Sirus Context establish sovereign identity
2	Upload Track (raw audio)	Save Draft — AES-256 encrypted, invisible to AI, no metadata required
3	Add Metadata later	Translations, TK Labels, Royalty Splits — at your own pace, over days, weeks, or years
4	Share with Manager / Label	Key Delegation — grant decryption access to Agent, Elder, or Tribe without exposing the raw key
5	Hit Release	Méh₁ns DVE Core wakes up, evaluates all Policy Packs, mints GovernanceToken
6	Song goes live	Data published to archive; AI pipeline can now access it — but only via the governance-cleared token

Where to Start

Read in this order. Do not skip steps.

1. Platform Integrity Map v1.0 — the normative superspec. Every rule in every repository flows from this document. Read Section 0 (Commercial Plugin Hostility) first. Then read Section 2 (Boot and Orchestration Sequence). Then the rest.
2. sparxstar-ouroboros-integrity specification — what you build first. Sprint 0. Nothing else begins until Ouroboros has passing tests. All other repositories depend on it.
3. The specification for the component you are building — Sirus, Helios, Sky, Mḗh₁n̥s, or Dheghom. Each has its own complete spec document.
4. The relevant schema — if you are building Sky Abilities or Dheghom modules, read the DVE schema repository to understand the conforming SCF shape.

The One Rule Before All Others

From the Platform Integrity Map, Section 0:

"No SPARXSTAR security, governance, sovereignty, or archival guarantee may depend on the behavior, restraint, or compatibility of third-party WordPress plugins. All such guarantees must be enforceable even when other plugins are present and operating normally."

You are not writing code for a clean controlled environment. You are writing code that must hold its guarantees while surrounded by arbitrary third-party behavior. Every enforcement mechanism in every repository exists because the environment cannot be controlled — only the code can.

The Sprint 0 Checklist

Before any component development begins, Ouroboros must ship and pass:

- All six exception classes defined and tested
- All four shared interfaces defined
- AgreementResult and ResourceSensitivity enums defined
- ContextPulse and GovernanceToken DTOs defined
- ValidationHelper complete — including normalize_unicode_nfc()
- Platform constants class complete
- 00-sparxstar-loader.php written with exit(1) hard stop
- shared-test-vectors.json written and both TypeScript and PHP AgreementEvaluator tests pass against it
- BootIntegrityException integration test passes — load order violation triggers exit(1)

When Sprint 0 is complete, the loader goes into mu-plugins/ and component development begins. The next document written for this system after Sprint 0 should be a commit message, not another spec.

Specification Documents

Each component has a complete specification document. These documents are the authoritative technical references for their respective repositories. The Platform Integrity Map governs all of them.

Specification	Covers	Status
Platform Integrity Map v1.0	All repositories — normative superspec, cross-component rules, closed open items, coding standards, toolchain	Complete
Ouroboros Integrity Spec v1.0	Execution substrate, loader, shared exceptions, contracts, DTOs, validation helper, constants, Sprint 0 build order	Complete
Sirus Context Engine Spec v3.0	Context engine, device continuity, trust engine, pulse generation, pulse verifier, device matcher, step-up policy, UEC migration	Complete
Helios Agreement Engine Spec v1.0	Edge + origin agreement evaluation, HeliosClientInterface, AgreementEvaluator, KV contract, session triple binding, audit log	Complete
Sky DVE Core Spec v1.0	Conversational intake, draft accumulation, commit gate, chat loop, session ownership, release gate, UUID system	Complete
Mếhạns DVE Core Spec v3.0	Epistemic Sieve, QUARANTINE semantics, Policy Registry, Policy Dispatcher, Governance Token, WordPress kernel integration, read interceptors	Complete
Dheghom DAL Spec v1.0	Modular DAL, 57 module field groups, entity/mapper/repository pattern, triple binding, token validation, schema engine	Complete

---

Ouroboros loads first. Ouroboros enforces last. Between those two moments, everything the platform promises is either kept or broken. Keep it.

Classes define behavior. Templates display it. JavaScript collects it. Only the system decides truth.